MOSAICOGROUP®
design and technology

TemplateBase Semantic_AI_Mapper

Contents

Indice **Error! Bookmark not defined.**

Executive summary 4

1. Contesto e motivazione 4

1.1 Il dominio (refrigerazione + supervisore + PLC) 4

1.2 Il problema (eterogeneita semantica) 4

2. Obiettivi del progetto 4

2.1 Obiettivo primario 4

2.2 Obiettivi secondari 5

3. Perimetro e fuori-perimetro 5

In perimetro 5

Fuori-perimetro (almeno nella prima fase) **Error! Bookmark not defined.**

4. Principi di progettazione (perche agentico qui e delicato) 5

5. Modello concettuale: Template Base, Dizionario, Knowledge Base 5

5.1 Template Base (canonico) 5

5.2 Dizionario (sinonimi -> concetto) 5

5.3 Knowledge Base (contesto e memoria) 5

6. Architettura proposta 6

6.1 Componenti 6

6.2 Decisione chiave: output azioni, non template completo **Error! Bookmark not defined.**

7. Requisiti non funzionali (NFR) 6

7.1 Sicurezza 6

7.2 Affidabilita e debug 6

7.3 Manutenibilita 6

7.4 Prestazioni 6

8. Flusso operativo end-to-end 6

9. Gestione casi difficili e regole del gioco 7

9.1 Variabile concettuale assente nel template 7

9.2 Termini non riconosciuti 7

9.3 Ambiguita persistente 7

9.4 Prevenzione loop e comportamenti instabili 7

10. Server MCP: responsabilita e API 7

10.1 Perche MCP e centrale 7

10.2 Tool/Endpoint minimi (prima release) 7

11. Roadmap di progetto (milestone aziendali) 8
12. Criteri di accettazione (Definition of Done) 8
13. Modalita operativa per il collega (come partire senza perdersi) 8
14. Allegati e dataset di test 8
15. Nota finale: cosa rende questo progetto professionale **Error! Bookmark not defined.**
16. Rischi e mitigazioni 8
16.1 Drift del dizionario 8
16.2 False positive 9
16.3 Vendor edge-cases 9
16.4 Loop agentico 9
16.5 Degrado prestazionale 9
17. Strategia di test e qualita 9
17.1 Tipi di test 9
17.2 Metriche e KPI 9
17.3 Non-regressione dizionario 9
18. Appendice: esempio PatchActions v1 9
19. Processo di rilascio e governance 11
20. Operational playbook (gestione operativa) **Error! Bookmark not defined.**

Executive summary

Questo progetto mira a costruire un sistema agentico, completamente locale, capace di normalizzare e aggiornare automaticamente i template presenti sui supervisori di refrigerazione. I template PRIME reali, generati in pratica dalla configurazione effettuata dal frigorista sui PLC (di marche diverse), contengono variabili (Read/Param/Virtual...) semanticamente simili ma rappresentate con nomi, registri e descrizioni non uniformi.

L'obiettivo e introdurre un livello di astrazione stabile attraverso un Template Base (canonico) e una Knowledge Base piu Dizionario che consenta di riconoscere concetti equivalenti e applicare aggiornamenti consistenti.

Il sistema viene eseguito in locale, utilizzando modelli su Ollama, e viene orchestrato tramite un server MCP che espone tool/endpoint controllati per leggere e scrivere template e device_list, validare output strutturati, gestire dizionario e knowledge base, e applicare patch ai template in modo sicuro e auditabile.

Il risultato atteso e un workflow che riduce il lavoro manuale e aumenta la standardizzazione, mantenendo pero garanzie aziendali: controllo, tracciabilita, riproducibilita, testabilita.

1. Contesto e motivazione

1.1 Il dominio (refrigerazione + supervisore + PLC)

Un punto vendita e dotato di un supervisore che si collega a piu PLC (banchi e celle). Ogni PLC puo essere di vendor diverso, con mappature registri e nomenclature differenti, pur descrivendo quasi sempre gli stessi concetti fisici (temperature, stati, sbrinamenti, ventilazioni, allarmi sonde...).

Nei template troviamo quindi: - ReadX: letture da registri PLC (variabili "vive") - ParamX: parametri e costanti operative (setpoint, soglie...) - Allarmi / Warnings: eventi e pre-eventi di criticita - VirtualVariables: logiche custom del frigorista (combinazioni di Read) - DataLoggerPen / DataLoggers: variabili storicizzate (timeseries) - AIVariables: variabili target o rilevanti per moduli predittivi (quando presente)

1.2 Il problema (eterogeneita semantica)

La difficolta non e "tecnica" ma semantica: un concetto come "Temperatura di Mandata" esiste praticamente sempre, ma puo essere descritto in decine di modi: temp.mandata, Sm, sonda 1, S1, delivery temperature, ecc. Inoltre il riferimento al registro (ReadX) cambia da vendor a vendor e da impianto a impianto.

Ne consegue che "capire i template" e standardizzarli richiede un sistema capace di riconoscere sinonimi e abbreviazioni, gestire ambiguita, accettare assenze (non tutti i PLC hanno tutte le sonde), ed evolvere nel tempo (nuovi modi di scrivere descrizioni).

2. Obiettivi del progetto

2.1 Obiettivo primario

Dato un template reale e una base di conoscenza, ricondurre le variabili e i metadati del template reale ai concetti del Template Base, applicando aggiornamenti consistenti (etichette, categorie, mapping, campi aggiuntivi) e producendo un output utilizzabile dal sistema esistente.

2.2 Obiettivi secondari

Automatizzare l'arricchimento della knowledge base partendo dal device_list del supervisore (vendor, famiglia, ruolo, feature flags). Rendere il processo autonomo, ma con guardrail: interazione umana solo se necessaria, stop conditions chiare e retry limitati. Rendere il sistema auditabile: log strutturati, report run-by-run, versioning del dizionario.

3. Perimetro e fuori-perimetro

In perimetro

- Normalizzazione e patch di metadati e mapping nei template.
- Gestione dizionario concetti/sinonimi.
- Enrichment knowledge base da device_list.
- Server MCP e contratti schema-first (Pydantic/JSON Schema).
- Logging, run report, e pipeline riproducibile.

4. Principi di progettazione (perche agentico qui e delicato)

Un agente LLM puo essere utile per gestire rumore linguistico e ambiguita, ma e rischioso se lasciato libero. Per questo il sistema deve essere progettato su principi enterprise-grade:

1**) Schema-first**: l'LLM deve produrre output strutturato che passa validazione rigorosa.

2) **Patch-based**, non rewrite-based: riscrivere template interi e fragile; meglio azioni piccole e reversibili. 3) Autonomia bounded: max retry, max step, stop conditions, escalation.

4) **Auditability come requisito**: ogni decisione deve essere ricostruibile (perche e stata fatta e con quale confidenza).

5. Modello concettuale: Template Base, Dizionario, Knowledge Base

5.1 Template Base (canonico)

Il Template Base rappresenta un set di concetti standard, eventuali categorie e metadati che desideriamo uniformare, ed eventuali campi aziendali aggiuntivi per rendere i template comparabili e processabili. Non e un template che sostituisce gli altri: e un riferimento semantico per garantire coerenza.

5.2 Dizionario (sinonimi -> concetto)

Il dizionario e la componente piu critica: connette lingua reale nei template (rumorosa, abbreviata) a concetti canonici. Deve essere versionato, auditato, aggiornabile come servizio MCP, compatibile con multilingua (it/en) e pattern.

5.3 Knowledge Base (contesto e memoria)

La KB include contesto operativo: informazioni derivate da device_list (vendor/famiglia), mapping gia risolti, regole di priorita (vendor hints), ed eventuali eccezioni note. L'obiettivo e ridurre ambiguita e aumentare determinismo nel tempo.

6. Architettura proposta

6.1 Componenti

A) **MCP Server (locale):** gatekeeper che controlla tool, validazione, logging e I/O, e impedisce azioni pericolose.
B) **Agent Orchestrator (Ollama)**: riceve obiettivi e contesto e produce un piano, una serie di PatchActions e (quando opportuno) richieste di aggiornamento dizionario.
C) **Matching Engine (ibrido)**: ranking a strati (normalizzazione, dizionario, fuzzy, fallback LLM, scoring e soglie).
D) **Report e Audit Layer**: log strutturati, run_report.json, versioning dizionario/KB.

7. Requisiti non funzionali (NFR)

7.1 Sicurezza

- Allowlist path: l'agente non deve scrivere fuori workspace.
- Nessuna esecuzione di codice arbitrario dal prompt.
- Sanitizzazione input (path traversal, ecc.).

7.2 Affidabilita e debug

1. Ogni run deve essere riproducibile (snapshot input e output).
2. Log strutturati JSONL con run_id, template_guid, step_id.
3. Error taxonomy (validation error, patch conflict, IO error...).

7.3 Manutenibilita

- **Moduli separati** (mcp server / schemas / matcher / agent loop).
- **Test minimi**: validazione schema, patch apply, matcher deterministico. Inoltre si deve dare la possibilità di modificare la knowledgebase a proprio piacimento. Il sistema deve permettere, per esempio, l'aggiornamento del TemplateBase di riferimoento senza modifiche ulteriori al sistema e garantire comunque il corretto funzionamento

7.4 Prestazioni

Operatività locale: minimizzare chiamate LLM; usare LLM solo dove serve. - Cache dei risultati di matching e delle normalizzazioni.

8. Flusso operativo end-to-end

a. **Load**: carica template reale, dizionario e KB, e opzionalmente template base.
b. **Extract e Normalize**: estrai descrizioni dai Values e normalizza testo.
c. **Match**: deterministico su dizionario, scoring; fallback LLM se sotto soglia o ambiguo.
d. **Produce PatchActions**: azioni strutturate con confidence, reason, evidence.
e. **Validate: Pydantic/JSON Schema**; se invalid -> self-repair bounded.
f. **Dry-run Patch**: produce diff e warning.
g. **Commit**: scrive output su directory run; no overwrite senza policy.
h. **Report**: run_report.json con mapping, ambigui, assenti, unmapped_terms e versioni.

9. Gestione casi difficili e regole del gioco

9.1 Variabile concettuale assente nel template

Caso previsto: non tutti i PLC hanno tutte le sonde/feature. Il sistema non deve fallire e deve registrare absent_concepts nel report.

9.2 Termini non riconosciuti

Se compaiono parole non presenti nel dizionario, devono finire in unmapped_terms. Il sistema deve poter proporre update dizionario tramite endpoint MCP, con audit e versioning.

9.3 Ambiguita persistente

Se due concetti risultano troppo simili, massimo N tentativi con contesto aggiuntivo. Se persiste, escalation a scelta umana motivata.

9.4 Prevenzione loop e comportamenti instabili

Retry limitati per step e per run, stop conditions chiare (es. 3 validation fail consecutivi), e tracciamento di ogni retry.

10. Server MCP: responsabilita e API

10.1 Perche MCP e centrale

Il server MCP limita cosa l'agente puo fare, impone validazione e logging, standardizza l'interfaccia dei servizi e rende il sistema integrabile e testabile.

10.2 Tool/Endpoint minimi (prima release)

Template:

- template.load(path),
- template.save(path, template)
- template.apply_patch(path, patch, dry_run)

Schema:

- schema.validate(schema_id, payload),
- schema.get(schema_id)

Dictionary:

- dictionary.search(text, lang),
- dictionary.upsert(entry),
- dictionary.bulk_suggest(terms)

KB:

- kb.load(),
- kb.save(versioned),

- kb.upsert_mapping(...)

Device list:

- device_list.enrich(path)

11. Roadmap di progetto (milestone aziendali)

Milestone A: fondazioni e standard.

Milestone B: MCP server + logging enterprise.

Milestone C: schema-first (Pydantic + JSON Schema).

Milestone D: dizionario e KB versionati.

Milestone E: matching engine ibrido.

Milestone F: agent loop autonomo bounded. Milestone G: device_list enrichment.

12. Criteri di accettazione (Definition of Done)

Una release e accettabile se ogni run genera output + report + log con run_id; PatchActions valida sempre o fallisce con errore leggibile; dry-run non produce eccezioni non gestite; dizionario aggiornabile e versionato; nessun loop (retry bounded); intervento umano solo su ambiguita persistenti e motivate.

13. Modalita operativa per il collega (come partire senza perdersi)

1) Studiare formato template e sezioni prioritarie. 2) Implementare core MCP e layer schema-first. 3) Poi matching deterministico. 4) Quando stabile, integrare fallback LLM. 5) Iterare sul dizionario guidati dai run_report (unmapped_terms -> update).

14. Allegati e dataset di test

Nel workspace sono presenti esempi di template e file correlati, da usare come set di regressione per matcher e patch e come base per costruire casi noti (ambiguo, assente, unmapped).

16. Rischi e mitigazioni

Questa sezione rende esplicito cosa puo andare storto e come il progetto lo gestisce.

16.1 Drift del dizionario

Descrizione: aggiornamenti frequenti senza governance possono creare duplicati, concetti sovrapposti e pattern troppo aggressivi. Mitigazioni: versioning forte, audit su upsert, regole anti-duplicato, CI di validazione collisioni, workflow di accettazione (slow path).

16.2 False positive

Descrizione: mapping errato con confidenza alta inquina analisi e standardizzazione. Mitigazioni: soglie conservative per auto-commit, evidence obbligatoria, blacklist abbreviazioni ambigue, dry-run sempre, modalita safe in caso di conflitti.

16.3 Vendor edge-cases

Descrizione: sezioni mancanti, nomi diversi (DataLoggerPen vs DataLoggers), descrizioni vuote. Mitigazioni: parser tollerante, alias mapping sezioni, report compatibilita, suite di regressione.

16.4 Loop agentico

Descrizione: retry infinito su validazione o patch conflict. Mitigazioni: retry bounded, stop conditions esplicite, run budget, logging retry.

16.5 Degrado prestazionale

Descrizione: usare troppo LLM rende lento e meno deterministico. Mitigazioni: LLM last, cache, batching, metriche llm_calls.

17. Strategia di test e qualita

17.1 Tipi di test

A) Schema e validazione: test unit deterministici.
B) Matching deterministico: unit + golden.
C) Patch apply: integration, idempotenza.
D) Golden templates regression: end-to-end su template reali rappresentativi.

17.2 Metriche e KPI

Metriche per run: mapped_count, ambiguous_count, unmapped_terms_count, llm_calls, avg_confidence, warnings_count.

KPI iniziali: riduzione unmapped_terms nel tempo, llm_calls stabile o in diminuzione, ambiguous_count ridotta per vendor noti.

17.3 Non-regressione dizionario

Ogni update dizionario deve triggerare: validazione schema, controlli collisioni sinonimi/pattern, esecuzione golden suite su subset di template.

18. Appendice: esempio di log

Quello che segue e un esempio realistico di output agentico strutturato. L'obiettivo e mostrare lo standard di qualita: reason chiara, evidence esplicita, confidence motivata, gestione unmapped e ambiguous.


```
{
  "schema_version": "v1",
  "run_id": "2026-01-16T14:22:11Z__b7c8c2a1",
```

```
  "template_guid": "e6a2260a-480d-d986-0048-87e3ef98d3c2",
  "source_files": {
    "template_path": "input/templates/e6a2260a-480d-d986-0048-87e3ef98d3c2.json",
    "dictionary_version": "v1.12.0",
    "kb_version": "v0.4.3"
  },
  "actions": [
    {
      "type": "map_variable",
      "section": "ContinuosRead",
      "source_key": "Read101",
      "target": {
        "concept_id": "temp_delivery",
        "category": "temperature",
        "labels": {
          "it": "Temperatura mandata",
          "en": "Delivery temperature"
        }
      },
      "patch": {
        "set_fields": {
          "Code": "Sm",
          "Measurement": "°C/°F",
          "Labels": [
            { "Code": "it", "Description": "Temperatura mandata (Sm)" },
            { "Code": "en", "Description": "Delivery temperature (Sm)" }
          ]
        }
      },
      "confidence": 0.92,
      "reason": "La descrizione contiene termini compatibili con il concetto di mandata (match su
sinonimi 'mandata', 'Sm'). Nessun altro concetto candidato supera la soglia.",
      "evidence": {
        "dictionary_entry_id": "temp_delivery",
        "matched_terms": ["mandata", "sm"],
        "normalization": {
          "raw_description": "Temp.mandata Sm",
          "normalized": "temp mandata sm"
        }
      }
    }
  ],
  "absent_concepts": [
    {
      "concept_id": "temp_return",
      "reason": "Non e presente alcuna Read con descrizioni compatibili; caso plausibile per questo
PLC/banco."
    }
  ],
  "ambiguous_matches": [
    {
      "section": "ContinuosRead",
      "source_key": "Read55",
      "candidates": [
```

```
        { "concept_id": "temp_evaporator", "confidence": 0.56 },
        { "concept_id": "temp_return", "confidence": 0.53 }
      ],
      "reason": "Descrizione corta e ambigua (es. 'S1'). Richiede contesto aggiuntivo o intervento
umano."
    }
  ],
  "unmapped_terms": [
    {
      "term": "evap",
      "context": {
        "section": "ContinuosRead",
        "source_key": "Read55",
        "raw_description": "Temp evap S1"
      },
      "suggested_action": "dictionary.suggest_or_upsert"
    }
  ],
  "validation": {
    "status": "ok",
    "warnings": [
      "DataLoggerPen section name may vary (DataLoggers). Ensure section alias mapping is enabled."
    ]
  }
}
```


19. Processo di rilascio e governance

Questa sezione definisce come promuovere cambiamenti in modo controllato, specialmente su dizionario e KB.

Versionamento: adottare un versionamento semantico o incrementale per dictionary e KB. Ogni versione deve essere associata a:

- changelog sintetico (cosa e cambiato)
- set di test eseguito (golden suite)
- firma/approvazione (anche leggera)

Flusso suggerito:

1) una run produce unmapped_terms e/o ambiguous_matches.

2) si propone update dizionario (pull request o file di patch dizionario).

3) si eseguono controlli automatici (schema + collisioni + subset golden suite).

4) se passa, si promuove a nuova dictionary_version e si registra nel registro revisioni.

Regole di rollback: mantenere sempre N versioni precedenti e la possibilita di forzare l'uso di una

Prospettive Future

1) Per migliorare lo score nella comparazione delle varaibli nel template col template base, potrebbe tornar utile far una comparazione nei comportamenti della variabile nelle time series, esempio: Mettiamo che in un template abbiamo una sonda chiamata Sonda1, non abbiamo idea di cosa registri, l'unico campo che ci può dar euna mano è l'unità di misura, ma ammettendo anche che ci sia una misura come "°C/F", non possiamo aver una chiara idea a che temperatura specifica stia facendo riferimento (se mandata, ripresa, sbrinamento etc...). L'unico altro modo per comprendere cosa sia, sarebbe quello di andare ad analizzare la sua timeseries effettuando un comparazione tra timeseries di default e quella interessata.
2) Sarebbe da esporre come tool al modello

Materiale di studio utile

1) MCP in 10 minutes

Breve introduzione a MCP (Model Context Protocol): a cosa serve e come funziona ad alto livello.
Link: https://www.youtube.com/watch?v=XIdQ6gO3Anc

2) Build Your Own Ai Agent Server in 10 Minutes

Video “hands-on” per costruire rapidamente un server per agenti AI (impostazione pratica in stile quickstart).
Link: https://www.youtube.com/watch?v=7aBRk_JP-qY&t=1257s

3) Pydantic (sito ufficiale)

Documentazione e riferimenti ufficiali di Pydantic (validazione dati, modelli, schema, ecc.).
Link: https://pydantic.dev/

4) MCP Python (Full Course)

Corso completo su MCP in Python (per implementazione, integrazione e utilizzo pratico).
Link: https://www.youtube.com/watch?v=VfZlglOWWZw&list=PLlrxD0HtieHjYfVUpGl_-ai7D6FRBjV-d

5) Instructor

Video intitolato “Instructor” (dettagli non recuperati qui).
Link: https://www.youtube.com/watch?v=p09yRj47kNM&t=904s

6) Come preparare i dati per gli LLM.

Breve video di Google che spiega come sistemare, preparare e comunicare con i LLMsa.
Link: https://www.youtube.com/watch?v=eihYrX7F7as

8)(Build ANYTHING With an Advanced MCP Server (Python, Authentication, Databases & More)

Come utilizzare gli MCP Server ad un livello avanzato
Link: https://www.youtube.com/watch?v=j5f2EQf5hkw&t=12s

9) Passing JSON to an LLM is SUPER wasteful

Ha effettivamente senso usare i JSON?

Link: https://www.youtube.com/shorts/YjPD9Alf1co

10) Masterclass sulla selezione di modelli linguistici di grandi dimensioni - novembre 2025

Come capire se un LLM fa al caso tuo o meno
Link: https://www.youtube.com/watch?v=n4NokjyAklg

11) Mathematics for Machine Learning Tutorial (3 Complete Courses in 1 video)

Per molti corsi di livello avanzato in Machine Learning e Data Science, è necessario rinfrescare le nozioni di base di matematica, argomenti che potresti aver studiato in precedenza a scuola o all'università, ma che sono stati insegnati in un altro contesto o in modo poco intuitivo, tanto che fai fatica a collegarli al loro utilizzo nell'informatica.

Questa specializzazione mira a colmare tale lacuna, consentendo di acquisire rapidamente le conoscenze matematiche di base, sviluppando una comprensione intuitiva e collegandole al Machine Learning e alla Scienza dei dati.

Al termine di questa specializzazione, avrete acquisito le conoscenze matematiche necessarie per proseguire il vostro percorso e seguire corsi più avanzati di Machine Learning.

Link: https://www.youtube.com/watch?v=0z6AhrOSrRs

12) Let's build GPT: from scratch, in code, spelled out.

Probabilmente il miglio video sui Traformers e Attention che puoi trovare in giro
Link: https://www.youtube.com/watch?v=kCc8FmEb1nY